Sujet de Projet / TP : Test & Validation Logicielle

Projet 1 : Qualification et Recette du Système CantineMaster

> ***Module :*** *Test et Validation des Logiciels (Assurance Qualité & QA Testing)*
> ***Niveau :*** *Première formation au test logiciel (L3 / M1 / Écoles d'Ingénieurs)*
> ***Durée estimée :*** *12 à 20 heures (en binôme ou individuel)*
> ***Livrables :*** *Dossier de recette + Scripts de tests automatisés Pytest + Fiches d'anomalies*

---

1. Contexte & Mise en Situation Professionnelle

Vous venez d'intégrer l'équipe Assurance Qualité (QA) d'une société éditrice de solutions logicielles éducatives en tant qu'**Ingénieur Test et Validation Junior**.
L'équipe de développement vient de finaliser la version 1.0.0 de **CantineMaster**, une application web Flask dédiée à la gestion de cantine scolaire, aux abonnements mensuels et au contrôle d'accès au réfectoire.

Votre mission est d'assurer la **recette logicielle complète** du système avant sa mise en production dans les établissements scolaires pilotes.
Vous devez vous appuyer rigoureusement sur le Cahier des Spécifications Fonctionnelles pour vérifier qu'aucune exigence n'est omise, déceler d'éventuels bugs et automatiser les tests de non-régression.

---

2. Missions à Réaliser

Mission 1 : Analyse des Exigences & Matrice de Traçabilité (RTM)

La première responsabilité d'un ingénieur test est de garantir la couverture exhaustive des besoins.
1. Étudiez le document `SPECIFICATIONS_FONCTIONNELLES.md` et identifiez toutes les règles de gestion (`RG-USR`, `RG-ETU`, `RG-ABO`, `RG-PAY`, `RG-FRQ`).
2. Construisez une **Matrice de Traçabilité des Exigences (Requirements Traceability Matrix)** sous forme de tableau associant :
- L'identifiant de la règle de gestion (ex : `RG-FRQ-02`).
- L'intitulé de la règle.
- Les identifiants des cas de test correspondants (ex : `TC-FRQ-001`, `TC-FRQ-002`).
- Le type de test (Unitaire, Intégration, Sécurité, IHM/Fonctionnel).
- Le niveau de criticité (Bloquant, Majeur, Mineur).

Mission 2 : Conception de Tests Boîtes Noires (Partitions d'Équivalence & BVA)

Appliquez les techniques formelles de test apprises en cours :

1. **Partitionnement en classes d'équivalence (Equivalence Partitioning) :**
2. Sur le montant d'un paiement (`RG-PAY-02` et `RG-PAY-03`) : identifiez les classes valides et invalides.
3. Sur le format du matricule étudiant (`RG-ETU-01`).
4. Sur la politique de complexité des mots de passe (`RG-USR-04`).
5. **Analyse des valeurs limites (Boundary Value Analysis - BVA) :**
6. Sur la période de validité d'un abonnement mensuel `[start_date, end_date]` (`RG-ABO-01`) :
   - Identifiez les 4 dates clés à tester (veille du début, jour de début, jour de fin, lendemain de la fin).
7. Sur la longueur du mot de passe (seuil de 8 caractères).
8. Sur le montant minimal de paiement (seuil de 0.00 € / 0.01 €).

Mission 3 : Table de Décision (Contrôle d'Accès au Réfectoire)

Le processus de contrôle d'accès à la cantine (`RG-FRQ-02` et `RG-FRQ-03`) combine plusieurs conditions logiques :
- Étudiant existant et actif ?
- Abonnement en cours de validité ?
- Statut de l'abonnement (`ACTIF` , `SUSPENDU` , `EN_ATTENTE`) ?
- Jour de la semaine autorisé par le forfait ?
- Repas déjà consommé sur ce service aujourd'hui ?

Travail demandé :
1. Rédigez une **Table de Décision complète** (Truth Table) listant l'ensemble des combinaisons de conditions possibles et les actions/résultats attendus (`ACCÈS AUTORISÉ` ou motif de refus précis).
2. Simplifiez la table pour éliminer les cas redondants (minimisation des tests).
3. Déduisez-en la liste minimale de cas de test à exécuter pour une couverture à 100%.

Mission 4 : Rédaction du Cahier de Recette

À partir du modèle fourni dans CAHIER_DE_RECETTE_ETUDIANTS.md :
1. Rédigez au moins **15 cas de tests détaillés** couvrant :
- La sécurité et le contrôle d'accès par rôle (RBAC).
- L'inscription des étudiants.
- La souscription et le renouvellement des abonnements.
- Les encaissements et quittances.

- La borne de pointage cantine.
2. Pour chaque cas de test, renseignez scrupuleusement :
- Identifiant unique (`TC-XXX-YYY`).
- Titre explicite et Exigence couverte (`RG-...`).
- Préconditions (état initial du système et de la base de données).
- Données de test (identifiants, dates, matricules précis).
- Étapes de reproduction numérotées.
- Résultat attendu (comportement système, message affiché, code HTTP, etc.).
- Statut d'exécution (Succès / Échec / Bloqué).

Mission 5 : Automatisation des Tests avec Pytest

Dans une approche DevOps / CI, les tests doivent être exécutables automatiquement à chaque commit.
1. Prenez en main l'arborescence des tests du projet sous `tests/` .
2. Exécutez la suite de tests existante :
`bash pytest -v`
3. Rédigez au moins **5 nouveaux tests automatisés** personnalisés dans un fichier `tests/test_etudiants.py` :
- Au moins 2 tests unitaires ciblant les services métier (`StudentService` , `SubscriptionService` , ou `PaymentService`).
- Au moins 1 test d'intégration HTTP simulant la connexion et une requête avec le client Flask (`client.post(...)`).
- Au moins 1 test négatif (vérifiant la levée d'une exception `ValueError` ou un statut HTTP 403).
4. Mesurez la couverture de code de vos tests :
`bash pytest --cov=app tests/`

Mission 6 : Signalement d'Anomalies (Fiches de Bugs IEEE 829)

Lors de vos séances de tests exploratoires ou d'exécution de cas limites :
1. Si vous identifiez un comportement non conforme aux spécifications ou une faille d'ergonomie, rédigez une **fiche d'anomalie normalisée** comprenant :
- Identifiant du bug (`BUG-001`).
- Titre synthétique.
- Sévérité (Bloquante, Critique, Majeure, Mineure) et Priorité (Haute, Moyenne, Basse).
- Environnement (OS, version Python, navigateur).
- Scénario de reproduction pas à pas.
- Résultat observé vs Résultat attendu.
- Capture d'écran ou extrait de trace d'erreur (stack trace).

---

3. Livrables Attendus & Modalités de Rendu

Vous rendrez une archive ZIP (ou lien vers un dépôt Git) contenant :
1. **Le Rapport de Recette Logicielle (PDF ou Markdown) :**
- Introduction et périmètre testé.
- La Matrice de Traçabilité Exigences <-> Tests.
- La Table de Décision de la borne de cantine.
- Le Cahier de Recette dûment complété avec statuts d'exécution.
- Les Fiches de Bugs identifiées (le cas échéant).
- Conclusion et avis qualité sur l'état de livraison du système.
2. **Le Code des Tests Automatisés :**
- Le fichier `tests/test_etudiants.py` correctement commenté et fonctionnel avec `pytest`.
- Une capture d'écran du résultat d'exécution de `pytest --cov`.

---

4. Grille d'Évaluation (Barème sur 20 points)

Critère d'Évaluation	Description	Points
Compréhension & Traçabilité (RTM)	Pertinence de la matrice de traçabilité, couverture complète des règles `RG-xxx` sans oubli.	/ 3 pts
Techniques Formelles de Test	Justesse des classes d'équivalence, identification rigoureuse des valeurs limites (BVA) et table de décision.	/ 4 pts
Qualité du Cahier de Recette	Clarté, rigueur et reproductibilité des cas de test (préconditions, étapes, données, résultats attendus).	/ 4 pts
Automatisation Pytest	Qualité du code Python, bon usage des fixtures, pertinence des assertions, exécution sans erreur.	/ 5 pts
Analyse & Fiches d'Anomalies	Qualité de l'argumentation, précision des fiches de bugs selon le standard IEEE 829.	/ 2 pts
Rapport & Professionnalisme	Clarté rédactionnelle, structure soignée, posture professionnelle d'ingénieur QA.	/ 2 pts
TOTAL		**/ 20 pts**

CantineMaster © 2026 — Document pédagogique officiel pour le cours de Test et Validation des Logiciels